Contadores

Electrónica II

Estudiante: Roldan Lucas Daniel

Carrera: Ing. Electrónica

Profesores:
Jorge Scandaliaris
Fernando Alberto Miranda Bonomi

November 9, 2026

Índice

1 Resumen **2**

2 Introducción **2**
2.1 Funcionamiento de cerrojo D 2
2.2 Funcionamiento de flip-flop D maestro-esclavo 3
2.3 Concepto de registro 4
2.4 Concepto de circuito secuencial 4
2.5 Concepto de circuito secuencial sincrónico 5
2.6 Concepto de circuito secuencial asincrónico 5
2.7 Máquinas de estado finito 5
2.7.1 Concepto 5
2.7.2 Estructura general y tipos 5
2.7.3 Formas de presentación 6
2.7.4 Asignación de estados 7
2.7.5 Análisis y síntesis 7

3 Materiales y Métodos **8**
3.1 Contador de tres bits 8
3.2 Contador de cuatro bits 9

4 Resultados **11**
4.1 Resultados del contador de 3 bits 11
4.2 Simulación del contador de 4 bits 12

5 Conclusiones **13**

6 Bibliografía **13**

1 Resumen

Este informe aborda el diseño e implementación de un contador secuencial síncrono de 4 bits en VHDL, fundamentado en el estudio de máquinas de estados finitos. En primer lugar, se analizó la teoría de latches, flip-flops D y registros, comparando el comportamiento de los sistemas secuenciales asíncronos frente a los síncronos. La arquitectura propuesta integra la lógica combinacional y síncrona mediante la técnica de datapath para ejecutar el conteo ascendente, la carga en paralelo, la retención de estado y el reset síncrono . Posteriormente, se realizó la verificación funcional mediante un banco de pruebas (*testbench*), evaluando la respuesta temporal con GHDL y observando las formas de onda en GTKWave. Finalmente, los resultados validaron la secuencia de conteo y la jerarquía de prioridades de control, demostrando la exacta coincidencia entre el diseño teórico y la simulación por software.

2 Introducción

El presente informe aborda el diseño, modelado y verificación funcional de un contador síncrono de 4 bits desarrollado en VHDL. A partir del estudio de los sistemas secuenciales y la teoría de máquinas de estado finito, se implementó una arquitectura basada en datapath que integra un registro de estado síncrono y lógica combinacional para conteo, carga paralela, retención y reset síncrono. La validez de la propuesta se constató mediante simulación por software utilizando GHDL para la compilación y GTKWave para el análisis gráfico de las formas de onda.

2.1 Funcionamiento de cerrojo D

En los circuitos Latch S-R (cerrojo S-R) se tienen inconvenientes cuando S y R cambian de estado al mismo tiempo, en general diseñar circuitos cuidando el *quién o cuándo* lleva mayores facilidades. Para este problema conviene implementar un latch D, como se muestra en la Figura 1, donde el *clk* se llama señal de reloj la cual es una señal cuadrada la cúal sirve para tener sincronia entre todo los circuitos implementados, la entrada D se llama señal de datos la cúal se considera el valor que que se desea guardar con el circuito Latch D.

Para entender su funcionamiento se ignora los valores que tienen de salida, en primera instancia se considera que el clk se encuentra en cero, se observa que para cualquier valor de D las salidas se quedan en sus valores previos puesto que el clk se conecta a las compuertas AND por lo tanto el nivel alto del clk marca cuando el circuito reacciona a la entrada D, luego se observa que si la entrada D esta en alto la del reset se queda en cero y el set se queda en alto por lo tanto el Latch almacena un uno (El mismo valor que la entrada D), al momento que D se pone a nivel bajo el reset se activa y el el Set se queda en estado bajo por lo tanto el Latch almacena un cero (El mismo valor que la entrada D). Con esté analisis se concluye la tabla de verdad de la Figura 1. Cuando el clk esta en nivel bajo se dice que el latch es opaco y cuando el clk se encuentra en nivel alto se dice que el latch es transparente.

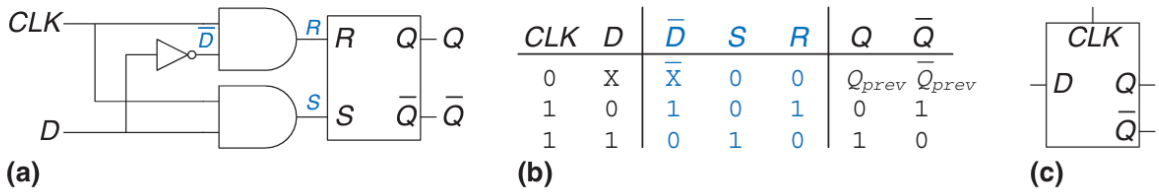


Figure 1: Latch D: (a) Esquemático, (b) tabla de verdad, (c) símbolo.

2.2 Funcionamiento de flip-flop D maestro-esclavo

El flip flop D se construye a base de dos circuitos Latch D con una señal clk complementarias como se ve en la Figura 2 , entre ellos el nodo de conexion se llama N1, el Latch L1 se llama maestro y el Latch L2 se llama esta esclavo. Cuando el clk se encuentra en estado bajo el latch maestro se encuentra transparente ante la entrada D, mientras que el segundo se encuentra opaco, por lo tanto el latch maestro toma el valor de C y lo almacena en su salida, cuando el clk cambia a nivel alto el maestro se vuelve opaco a D y el esclavo se vuelve transparente a su D y toma el valor setedo previamente. El fliflop D toma el valor de D guarda el valor de D hasta que se encuentra con otro cambio del reloj.

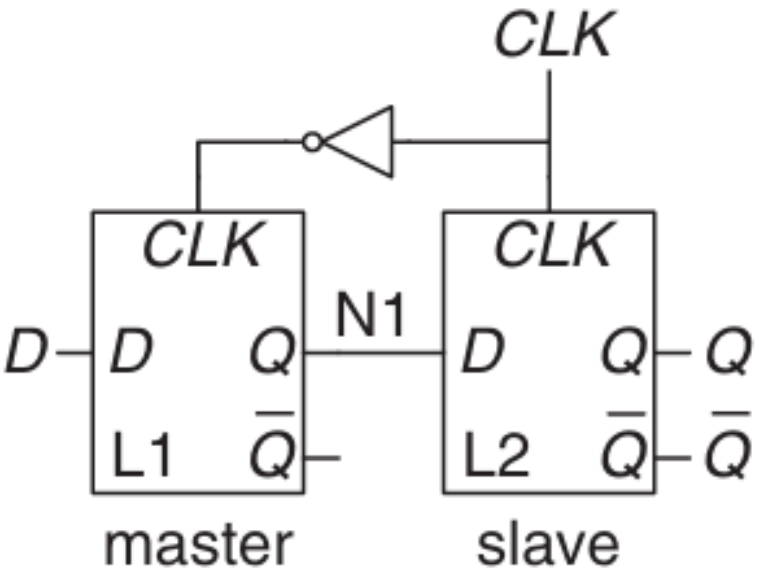


Figure 2: Estructura de un flip-flop D maestro-esclavo.

2.3 Concepto de registro

Los registros de n bits están formados por flip-flops que comparten una única señal de reloj (CLK). Esto permite cargar todos los flip-flops al mismo tiempo. Su función es almacenar una cadena de bits en paralelo. En la Figura 3 se muestra el circuito esquematico y su simboló, en este se representa los buses de datos como las lineas que entran y que tienen una diagonal cortando junto al número n de bits que entran.

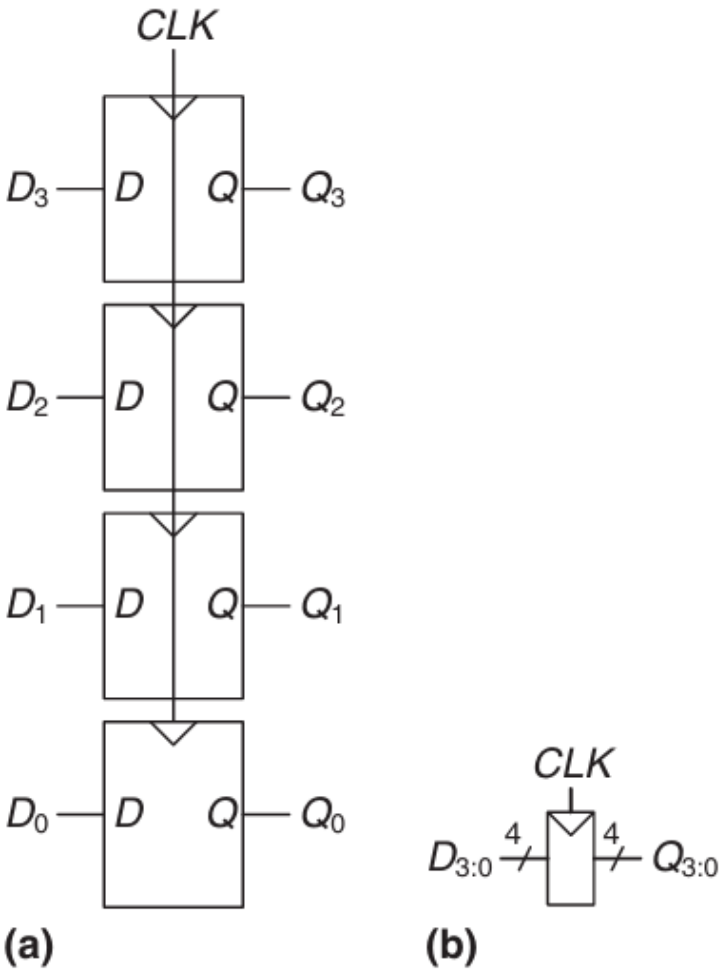


Figure 3: Registro: (a) Esquemático, (b) símbolo.

2.4 Concepto de circuito secuencial

Los circuitos de lógica secuencial tienen salidas que dependen de los valores que toman sus entradas en el presente y en el pasado, por lo tanto necesitan memorias. Se puede abstraer los valores previos en el concepto de estados del sistema, el estado de un circuito secuencial digital es un conjunto de bits llamados variables de estado que contienen toda la información sobre el pasado necesaria para explicar el comportamiento futuro del circuito. Ejemplos de estos circuitos son Latches o Flip-flops.

2.5 Concepto de circuito secuencial sincrónico

Un circuito secuencial sincrónico se compone de combinaciones de circuitos tales como:

- Al menos un registro en todo el circuito.
- Cada componente es un registro o un circuito de lógica combinacional.
- Entre cada lazo ciclico debe haber al menos un registro.

Cabe mencionar que los registros de estos circuitos deben estar todos al mismo clk. Ejemplo de circuitos sincronicos son los registros.

2.6 Concepto de circuito secuencial asincrónico

Los circuitos secuenciales asincrónicos no utilizan una señal de reloj global para coordinar sus transiciones. Sus celdas de memoria (como los latches SR) responden de inmediato a los cambios en sus entradas. Al carecer de temporización fija, procesan los eventos tan pronto como la información está disponible. Su diseño exige un control riguroso para evitar condiciones de carrera causadas por los retardos de propagación. Son esenciales para implementar elementos de memoria primarios y procesar señales o interrupciones externas impredecibles.

2.7 Máquinas de estado finito

2.7.1 Concepto

Una máquina de estados finitos (FSM) es un circuito secuencial con un número acotado y definido de estados de memoria posibles. Cambia de un estado presente a uno siguiente en cada ciclo de reloj, según sus entradas actuales y sus estados previos. Se compone internamente de un registro de estado (flip-flops), lógica combinacional de estado siguiente y lógica de salidas. Según el modelo, sus salidas dependen únicamente del estado actual (Moore) o del estado y las entradas presentes (Mealy). Constituye el cerebro o unidad central de control para coordinar secuencias y procesos temporizados en sistemas digitales.

2.7.2 Estructura general y tipos

La estructura general de una maquina de estado finito se compone de tres bloques:

- Logica combinacional de estado siguiente: Toma las entradas y el estado actual y entrega el estado siguiente.
- Registro: Conjunto de flip-flops que se encargan de preservar el estado anterior de la maquina.
- Logica combinacional de salida: Entrega las señales de salida.

Existen dos tipos principales las maquinas de Moore y las maquinas de Mealy, en la Figura 4, en la maquina de Moore la salida solo depende del estado presente, sus salidas cambian unicamente con flancos de reloj lo que evita glitches u osilaciones. Por otro lado en la maquina de Mealy su salida depende del estado presente y de las entradas del sistema, estas suele requerir menos estados que las maquinas de Moore por lo que son más simples.

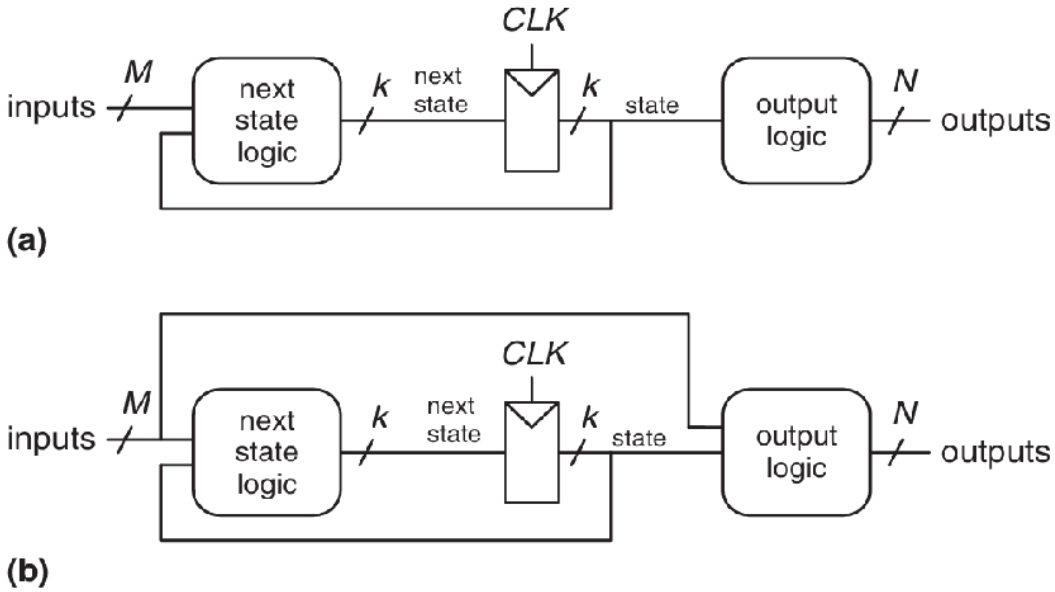


Figure 4: Máquinas de estado finito: (a) máquina de Moore, (b) máquina de Mealy.

2.7.3 Formas de presentación

Una máquina de estados finitos se puede representar mediante un diagrama de estados, como se muestra en la Figura 5, en esté digrama las flechas representan transiciones y los circulos o nodos representan los estados. En FSM de Moore las salidas se representan junto a los estados en los circulos y en las FSM de Mealy sobre la flecha de transición se escribe Entrada/salida.

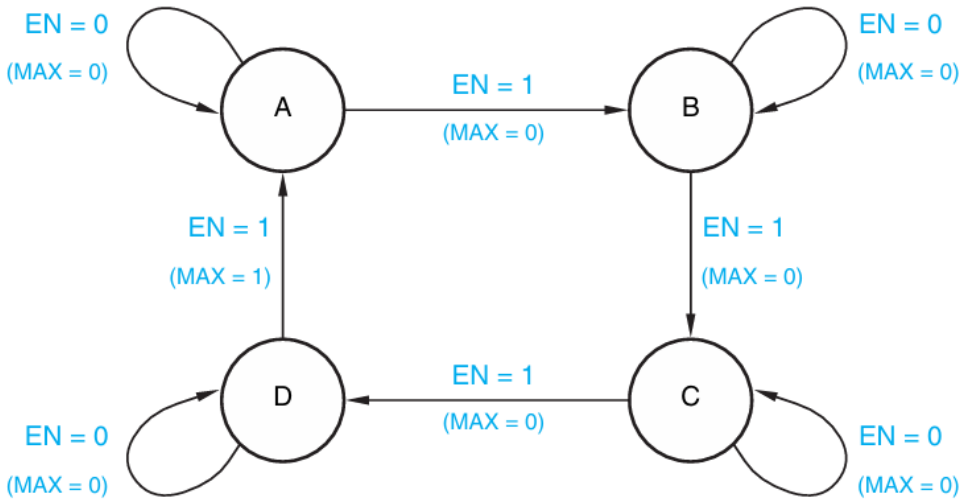


Figure 5: Diagrama de estados finitos con cuatro estados y transiciones etiquetadas.

Otra forma son las tablas de estados las cuales son una matriz que lista ordenadamente los

estados presentes, las combinaciones de entrada posibles, los estados siguientes resultantes y las salidas correspondientes.

2.7.4 Asignación de estados

La asignación de estados (o codificación de estados) consiste en traducir los estados simbólicos abstractos de un sistema ($S_0, S_1, S_2, \dots$) a combinaciones concretas de variables binarias que representan el contenido de los elementos de memoria (flip-flops).

Se pueden emplear distintos códigos binarios según la aplicación, como la codificación binaria natural (por ejemplo, $S_0 \to 00$ y $S_1 \to 01$), el código Gray o la codificación One-Hot (donde se asigna un flip-flop exclusivo a cada estado).

2.7.5 Análisis y síntesis

El **análisis de una FSM** es el proceso aproximadamente inverso a la síntesis [4]. Su objetivo es determinar la función lógica y el comportamiento temporal a partir del esquema de un circuito secuencial dado o de sus ecuaciones de excitación [4, 5].

El procedimiento estándar de análisis comprende los siguientes pasos:

1. **Obtención de las ecuaciones de excitación:** Se derivan las ecuaciones lógicas del estado siguiente y de salida a partir del circuito o diagrama lógico existente [4, 6].

2. **Construcción de la tabla de verdad y transición:** Se evalúan las ecuaciones considerando todas las combinaciones posibles de entradas externas y estados actuales para determinar los valores del estado futuro [5, 7].

3. **Reconstrucción del modelo simbólico:** Se reemplazan los patrones binarios por sus estados simbólicos equivalentes para generar la tabla o el diagrama de estados (de burbujas), lo que permite interpretar qué función o secuencia ejecuta el sistema [7, 8].

La síntesis de una FSM es el proceso estructurado de diseño que parte de una especificación o problema abstracto y conduce a la implementación de la lógica combinacional y secuencial [6, 9]. El proceso sistemático de síntesis comprende los siguientes pasos:

1. **Construcción del modelo abstracto:** Elaborar el diagrama o la tabla simbólica de estados que describe el comportamiento requerido ante cada entrada [9].

2. **Asignación y codificación de estados:** Determinar el número de flip-flops necesarios y asignar patrones de bits a cada estado simbólico [2, 3].

3. **Generación de la tabla de transición:** Construir la tabla de verdad binaria sustituyendo los estados simbólicos por sus combinaciones correspondientes de variables de estado [10, 11].

4. **Elección del tipo de flip-flop:** Seleccionar el dispositivo de memoria que se utilizará, habitualmente flip-flops D [6, 11].

5. **Derivación y minimización de ecuaciones:** Minimizar la lógica combinacional mediante mapas de Karnaugh o herramientas de síntesis de VHDL, para obtener las funciones optimizadas del estado siguiente (F_{sig}) y de las salidas (Z) [6].

3 Materiales y Métodos

3.1 Contador de tres bits

Para el desarrollo del contador se representó lo solicitado mediante un diagrama de estados y una tabla de estados. A partir de estas representaciones se define la codificación y se construye la tabla de transición, que permite sintetizar la lógica de salida y la lógica del estado siguiente. En la Figura 6 se muestran el diagrama y la tabla correspondientes.

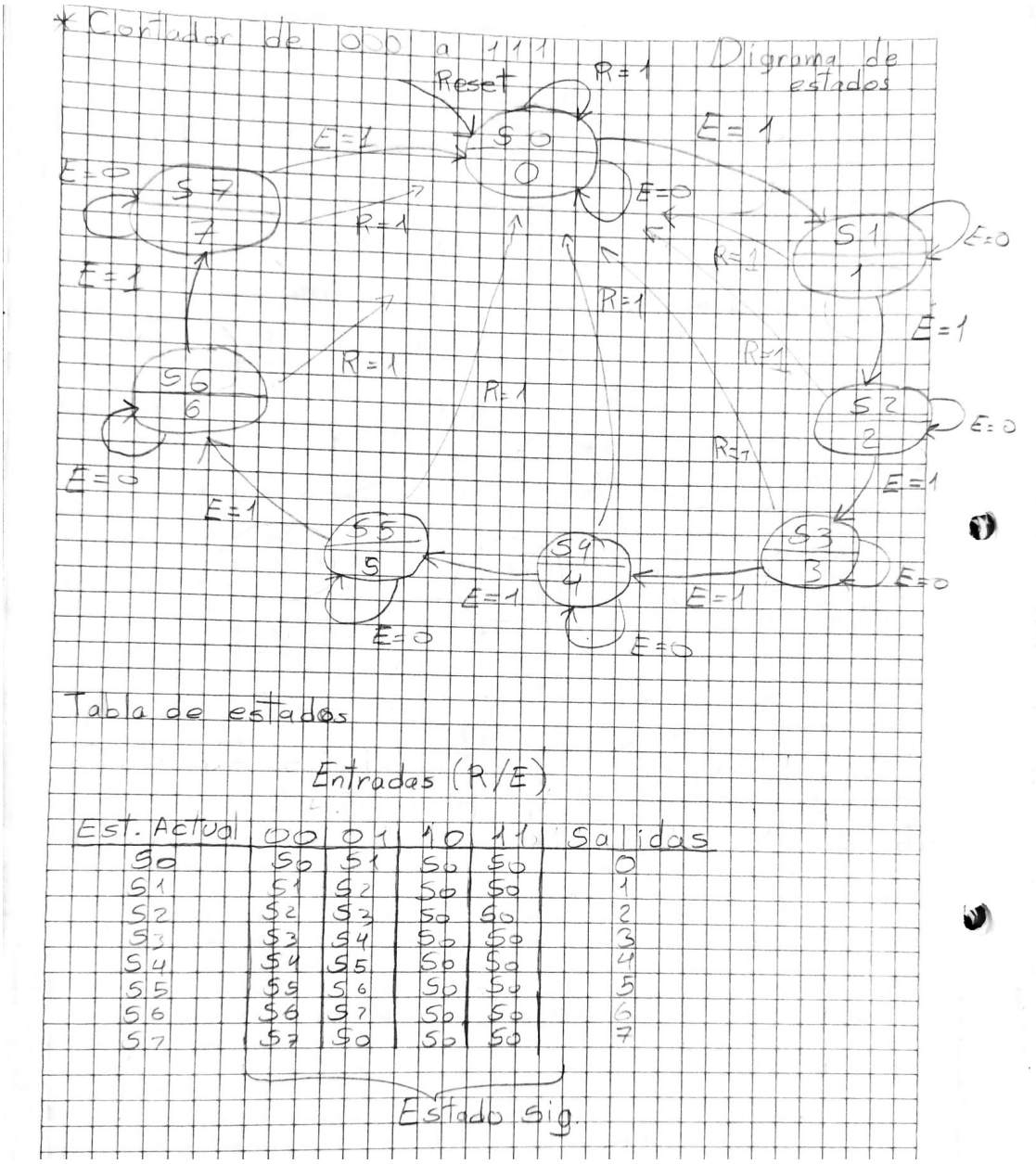


Figure 6: Diagrama de estados y tabla de estados del contador de tres bits.

Para la codificación de estados se optó por elegir binario natural para los estados de la máquina y sus salidas. Luego, se reescribe la tabla de estados con la codificación utilizada para

obtener la tabla de transición, como se muestra en la Figura 7. A partir de esta tabla se opta por utilizar flip-flops tipo D, cuya ecuación característica tiene una relación directa con la entrada.

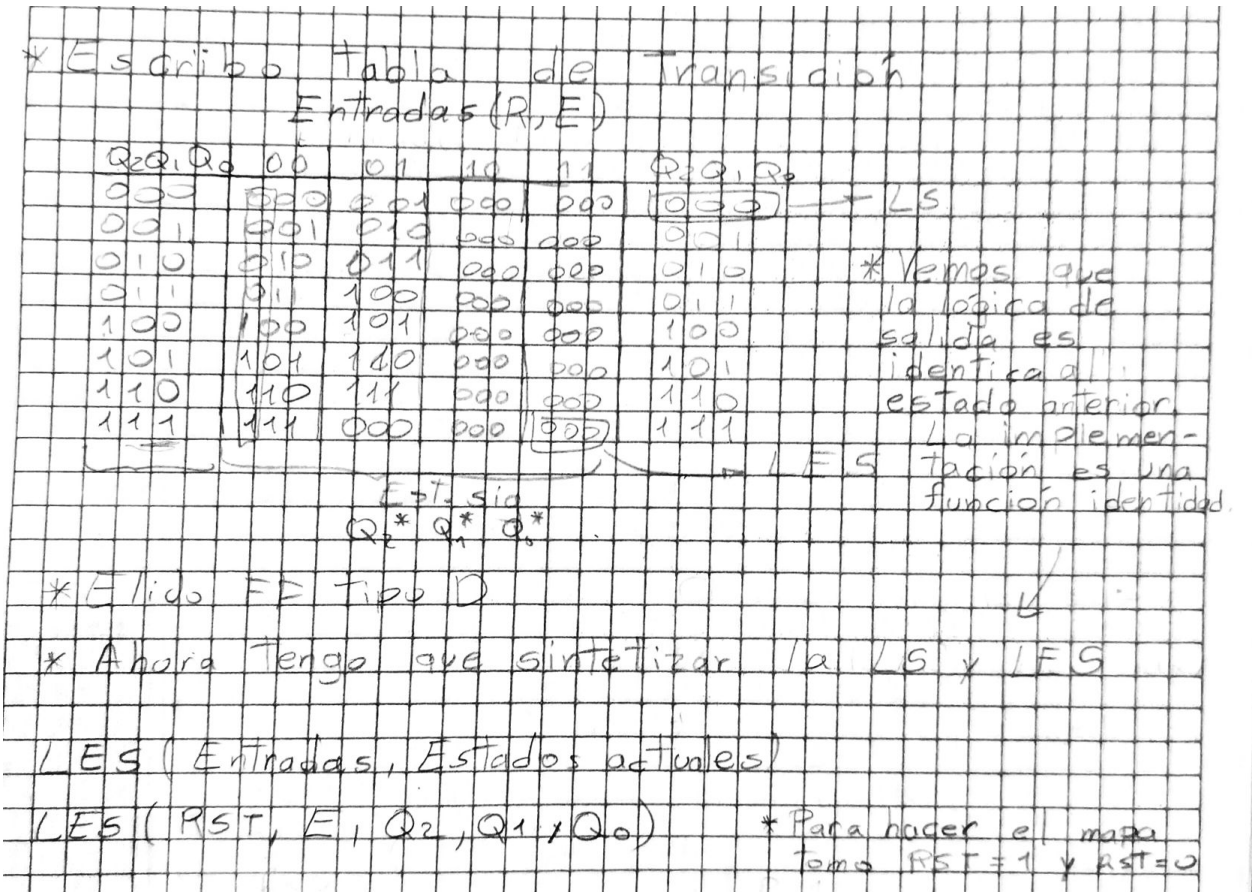


Figure 7: Tabla de transición del contador de tres bits con codificación binaria natural.

3.2 Contador de cuatro bits

El *datapath* es la parte del circuito que almacena y procesa los datos. En este contador, el registro de cuatro bits conserva el valor actual y entrega ese estado como salida. La lógica combinacional calcula el siguiente valor —por ejemplo, incrementándolo— y selecciona qué dato cargar en el registro según las señales de control. En el diseño se consideran el conteo, la carga en paralelo, la retención del valor y el reset síncrono. El registro actualiza su contenido en el flanco activo del reloj. La Figura 8 representa la organización propuesta.

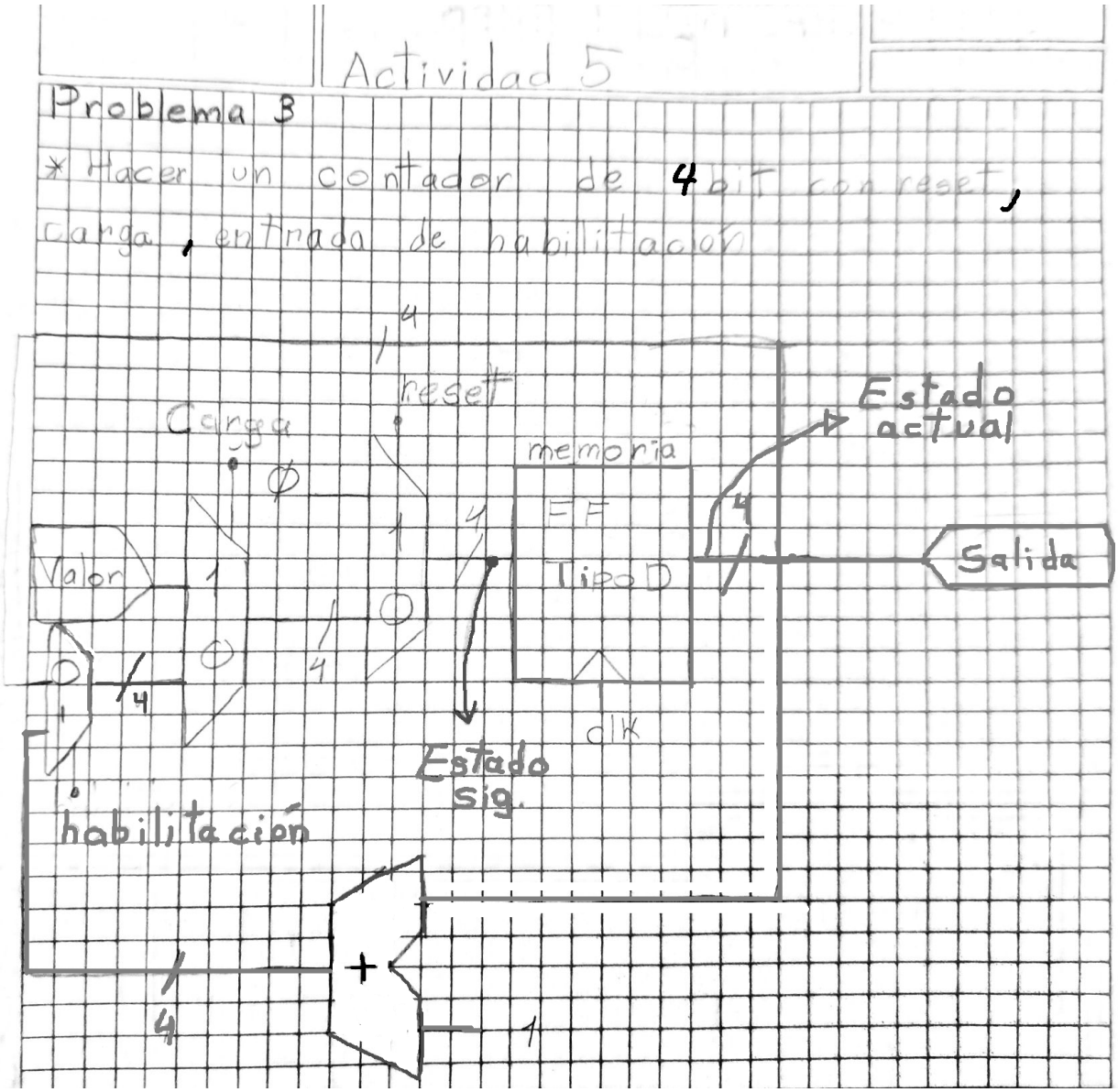


Figure 8: Datapath propuesto para el contador síncrono de cuatro bits.

Para el desarrollo, simulación y verificación funcional de los diseños secuenciales, se empleó un entorno de desarrollo basado en software libre.

- *Lenguaje de Descripción de Hardware (VHDL)*: Se utilizó para la descripción estructural y comportamental de los circuitos digitales (cumpliendo estándares VHDL-2008).
- *GHDL*: Compilador y simulador de VHDL utilizado para el análisis sintáctico, la elaboración jerárquica de la entidad y la ejecución de la simulación por software.
- *GTKWave*: Visor gráfico de formas de onda utilizado para la inspección y verificación temporal de las señales internas y externas exportadas en formato .ghw.
- *GNU Make & MSYS2*: Herramienta de automatización para gestionar el flujo de compilación e integración mediante reglas en un archivo Makefile.

- *Visual Studio Code*: Editor de texto y entorno de desarrollo integrado para la codificación de los archivos .vhd.

4 Resultados

4.1 Resultados del contador de 3 bits

El circuito sintetizado se implementa con lógica combinacional y tres flip-flops tipo D. Las compuertas calculan las entradas de los flip-flops a partir del estado actual y de la señal de habilitación; los flip-flops almacenan el estado y lo actualizan en el flanco activo del reloj. En la Figura 9 se presenta el circuito lógico obtenido.

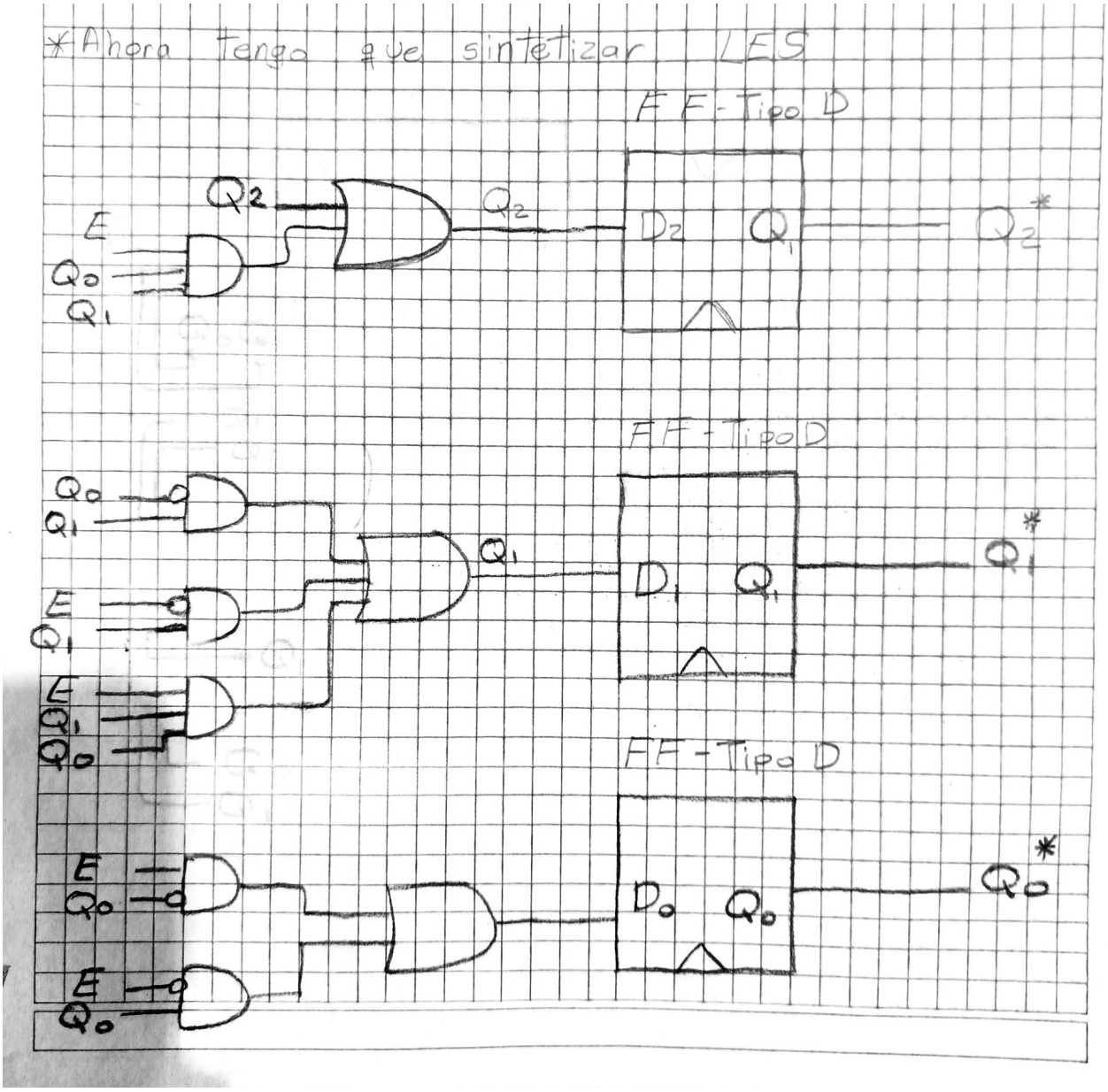


Figure 9: Circuito lógico sintetizado del contador de tres bits con flip-flops tipo D.

El diagrama de la Figura 10 muestra la evolución temporal del reloj, el reset, la habilitación y las salidas Q_0, Q_1 y Q_2. Permite comparar las transiciones de estado con los flancos de reloj y observar cómo las señales de control afectan el funcionamiento del contador.

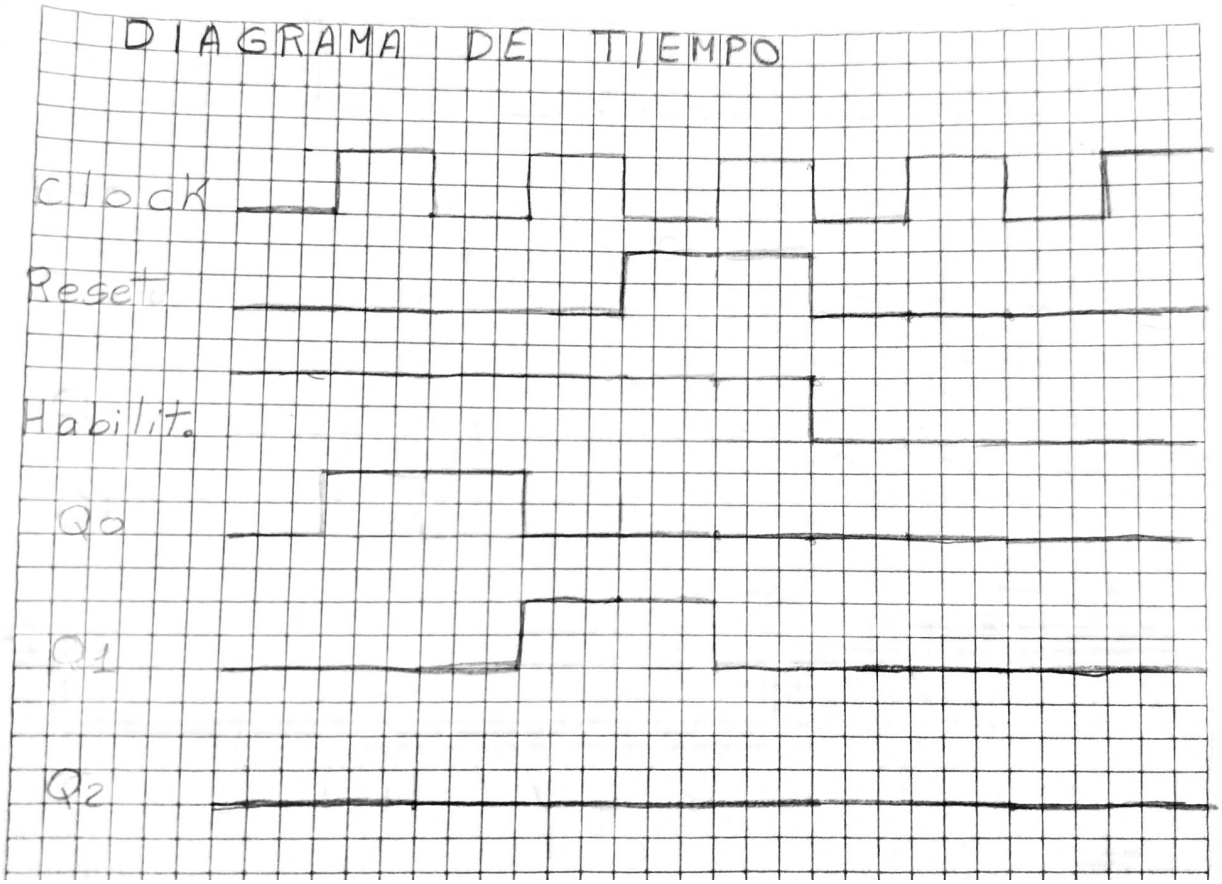


Figure 10: Diagrama temporal del contador de tres bits y sus señales de control.

4.2 Simulación del contador de 4 bits

La Figura 11 muestra las formas de onda observadas en GTKWave para las señales de reloj, carga, habilitación, reset, valor de entrada y salida. En la simulación, la salida recorre los valores 0, 1, 2 y 3; luego carga el valor 10 y continúa el conteo en 11 y 12. Esto permite verificar visualmente las operaciones de conteo y carga paralela del contador.

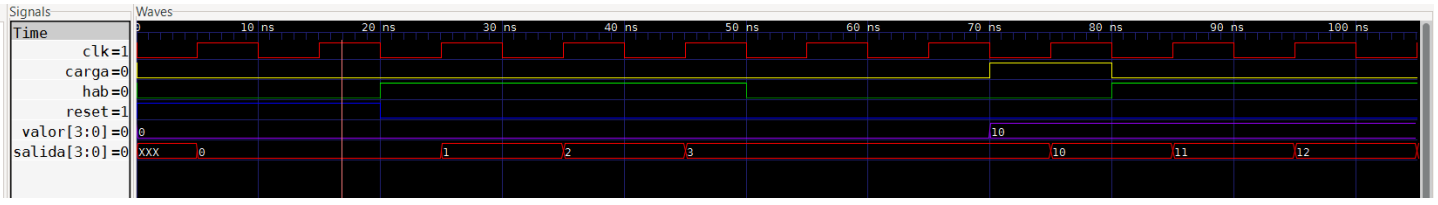


Figure 11: Simulación del contador de cuatro bits observada en GTKWave.

5 Conclusiones

El desarrollo permitió aplicar los conceptos de lógica secuencial y máquinas de estado finito al diseño de un contador síncrono ascendente de 4 bits en VHDL. La separación entre el registro de estado y la lógica combinacional facilitó organizar las operaciones de conteo, carga paralela, retención y reset síncrono.

De acuerdo con las simulaciones realizadas con GHDL y el análisis de las formas de onda en GTKWave, se verificó la secuencia de conteo y la respuesta del circuito ante las señales de control, respetando la jerarquía de prioridades definida para el diseño. Esto muestra la utilidad de combinar el modelado en VHDL con la simulación para detectar y evaluar el comportamiento del sistema antes de una posible implementación física.

Como trabajo futuro, se propone sintetizar e implementar el diseño en una FPGA para comprobar su funcionamiento en hardware y comparar las mediciones experimentales con los resultados de simulación.

6 Bibliografía

References

[1] Harris, S. y Harris, D. (2021). *Digital Design and Computer Architecture, RISC-V Edition* (1.ª ed.). Morgan Kaufmann.

[2] Wakerly, J. F. (2018). *Digital Design: Principles and Practices* (5.ª ed.). Pearson.